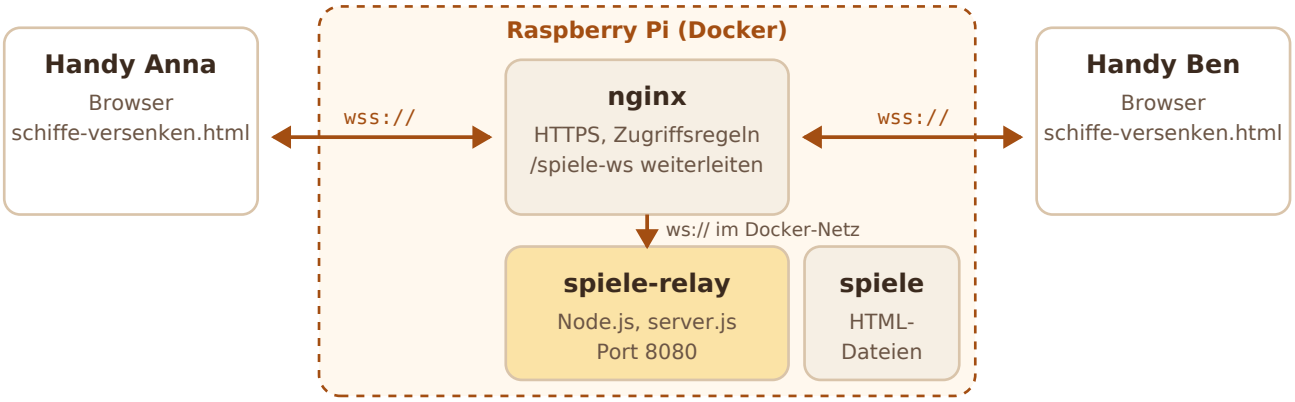
Der Weiterleitungsdienst spiele-relay

Node.js und JavaScript erklärt am Code von Schiffe versenken – für alle, die programmieren können, aber JavaScript noch nie benutzt haben

1. Worum es geht

Bei Schiffe versenken spielen zwei Personen auf zwei Handys gegeneinander. Die Handys können sich nicht direkt erreichen, also braucht es jemanden in der Mitte, der die Spielzüge weiterreicht. Das ist der **Weiterleitungsdienst** `spiele-relay`: ein kleines Programm von 150 Zeilen, geschrieben in **JavaScript** und ausgeführt mit **Node.js**, verpackt in einen Docker-Container auf deinem Pi.



Wichtig für das Verständnis: Der Dienst ist absichtlich **dumm**. Er kennt keine Spielregeln, sieht keine Schiffe und speichert nichts auf der Festplatte. Er macht genau drei Dinge:

Aufgabe	Wie
Zwei Geräte pro Spiel zusammenbringen	Jedes Spiel hat einen Code (z. B. k7m2x9ab) mit zwei Plätzen: Gastgeber und Gast.
Nachrichten weiterreichen	Was Anna schickt, bekommt Ben – unverändert.
Missbrauch verhindern	Fremde abweisen, Nachrichtenflut stoppen, tote Verbindungen aufräumen.

Die Spiellogik und alle Prüfungen gegen Schummeln stecken in der HTML-Datei, also im Browser. Dieses Dokument erklärt erst die Sprache an kleinen Beispielen (Kapitel 2) und geht dann den echten Code Abschnitt für Abschnitt durch (Kapitel 3–5).

Alle Beispiele sind echt: Jedes Beispiel in diesem Dokument wurde beim Erstellen mit Node.js 22 ausgeführt; die grünen Kästen zeigen die tatsächliche Ausgabe. Die Code-Ausschnitte aus `server.js` sind direkt aus der Datei übernommen, mit ihren echten Zeilennummern.

2. JavaScript und Node.js in Kurzform

JavaScript ist die Sprache, die jeder Browser versteht. **Node.js** ist ein Programm, das JavaScript außerhalb des Browsers ausführt – zum Beispiel als Server. Man startet ein Skript wie in Python: `node server.js` statt `python server.py`.

Im Folgenden die Sprachelemente, die im Weiterleitungsdienst vorkommen – jeweils mit kleinem Beispiel und, wo es hilft, dem Vergleich zu Python.

2.1 Variablen: const und let

`const` ist eine Variable, die nicht neu zugewiesen werden kann (vergleichbar mit `final` in Java). `let` ist eine normale Variable. Typen werden nicht angegeben. Texte in Backticks (`` `...` ``) können mit `${...}` Werte einsetzen – wie f-Strings in Python.

Beispiel

```
const name = "Pi";
let zaehler = 0;
zaehler++;                        // wie zaehler += 1
console.log(`Hallo ${name}, Zähler: ${zaehler}`);   // console.log = print
```

Ausgabe

```
Hallo Pi, Zähler: 1
```

2.2 Vergleiche: === statt ==

JavaScript hat zwei Gleichheitsoperatoren. `==` wandelt Typen vorher um und ist deshalb tückisch. Im Code wird ausschließlich `===` („gleich und gleicher Typ“) und `!==` verwendet.

Beispiel

```
console.log(1 == "1");    // Zahl und Text werden angeglichen
console.log(1 === "1");   // strenger Vergleich
console.log("gast" === "gast");
```

Ausgabe

```
true
false
true
```

2.3 Funktionen und Pfeilfunktionen

Funktionen gibt es in zwei Schreibweisen. Die kurze **Pfeilfunktion** `x => ...` entspricht Pythons `lambda x: ...`. Der Ausdruck `a ? b : c` ist ein Inline-if (`b if a else c` in Python). Genau so ist `anderer` in `server.js` Zeile 17 geschrieben:

Beispiel

```
function doppelt(x) {
  return x * 2;
}
const anderer = rolle => (rolle === "gast" ? "gastgeber" : "gast");

console.log(doppelt(21));
console.log(anderer("gast"), anderer("gastgeber"));
```

Ausgabe

```
42
gastgeber gast
```

2.4 Wahr, falsch und das || für Standardwerte

In Bedingungen gelten `0`, `""` (leerer Text), `null`, `undefined` und `NaN` als falsch, alles andere als wahr. Darauf baut ein häufiger Trick auf: `a || b` liefert `a`, wenn `a` „wahr“ ist, sonst `b`. So setzt der Server Standardwerte. Mit `!!wert` macht man aus einem beliebigen Wert ein echtes `true`/`false`.

Beispiel

```
// process.env enthält die Umgebungsvariablen; nicht gesetzte sind undefined
console.log(Number(undefined) || 8080);   // Variable fehlt -> Standardwert
console.log(Number("9000") || 8080);      // Variable gesetzt -> ihr Wert
console.log(!!"", !!"abc", !!null);
```

Ausgabe

```
8080
9000
false true false
```

2.5 Objekte, Map und Destrukturierung

Ein **Objekt** `{ a: 1 }` ist wie ein Python-Dictionary mit Text-Schlüsseln; auf Felder greift man mit `obj.a` oder `obj["a"]` zu. Eine `Map` ist ein echtes Wörterbuch mit `get`, `set`, `delete` und `size`. **Destrukturierung** packt Felder direkt in Variablen aus – fehlt ein Feld, ist es `undefined` (ähnlich `None`).

Beispiel

```
const spiele = new Map();
spiele.set("k7m2x9ab", { zuletzt: 1700000000000 });
console.log(spiele.size, spiele.get("k7m2x9ab").zuletzt, spiele.get("gibtsnicht"));

const rolle = "gast";
const socket = { gastgeber: "Annas Verbindung", gast: null };
console.log(socket[rolle], socket["gastgeber"]);   // Feldname aus einer Variablen

const { spiel, fehler } = { spiel: "das Spiel" };   // Destrukturierung
console.log(spiel, fehler);
```

Ausgabe

```
1 1700000000000 undefined
null Annas Verbindung
das Spiel undefined
```

2.6 Schleifen

`for (const x of liste)` entspricht Pythons `for x in liste`. Bei einer Map bekommt man Paare, die man direkt auspacken kann.

Beispiel

```
for (const rolle of ["gastgeber", "gast"]) console.log(rolle);

const spiele = new Map([["code1", "Spiel A"], ["code2", "Spiel B"]]);
for (const [code, spiel] of spiele) console.log(code, "->", spiel);
```

Ausgabe

```
gastgeber
gast
code1 -> Spiel A
code2 -> Spiel B
```

2.7 Texte prüfen: reguläre Ausdrücke

`/^[a-z0-9]{8}$/` ist ein regulärer Ausdruck direkt im Code (in Python: `re.compile(r"^[a-z0-9]{8}$")`). `.test(text)` liefert `true` oder `false`. Damit prüft der Server Spielcodes und Geheimwerte.

Beispiel

```
const istCode = text => /^[a-z0-9]{8}$/.test(text);
console.log(istCode("k7m2x9ab"));     // genau 8 Kleinbuchstaben/Ziffern
console.log(istCode("K7M2X9AB"));     // Großbuchstaben nicht erlaubt
console.log(istCode("k7m2"));         // zu kurz
```

Ausgabe

```
true
false
false
```

2.8 JSON

Nachrichten zwischen Browser und Server sind Text im JSON-Format. `JSON.stringify` macht aus einem Objekt Text, `JSON.parse` wieder ein Objekt (wie `json.dumps`/`json.loads` in Python).

Beispiel

```
const nachricht = { typ: "schuss", runde: 1, x: 3, y: 4 };
const text = JSON.stringify(nachricht);
console.log(text);
console.log(JSON.parse(text).x);
```

Ausgabe

```
{"typ":"schuss","runde":1,"x":3,"y":4}
3
```

2.9 Das Wichtigste: Ereignisse statt Warten

Hier unterscheidet sich Node.js am stärksten von klassischem Code. Ein Node-Programm läuft in **einem einzigen Thread** und wartet nie aktiv. Stattdessen sagt man: „Wenn Ereignis X passiert, rufe diese Funktion auf.“ Das geschieht mit `.on("ereignis", funktion)`. Node.js ruft die Funktion später auf, wenn das Ereignis eintritt – das Programm läuft derweil weiter.

Beispiel

```
const { EventEmitter } = require("events");
const verbindung = new EventEmitter();

verbindung.on("message", text => console.log("Nachricht erhalten:", text));
verbindung.on("close", () => console.log("Verbindung zu"));

verbindung.emit("message", "Hallo");   // so löst Node.js die Ereignisse intern aus
verbindung.emit("close");
```

Ausgabe

```
Nachricht erhalten: Hallo
Verbindung zu
```

Auch Zeitgeber funktionieren so: `setTimeout(f, ms)` ruft `f` einmal nach `ms` Millisekunden, `setInterval(f, ms)` immer wieder. Die Reihenfolge zeigt, dass nichts blockiert:

Beispiel

```
console.log("A");
setTimeout(() => console.log("C – kommt erst, wenn alles andere erledigt ist"), 0);
console.log("B");
```

Ausgabe

```
A
B
C – kommt erst, wenn alles andere erledigt ist
```

Warum das für einen Server ideal ist: Ein einziger Thread kann Tausende Verbindungen bedienen, weil er nie auf eine wartet. Und weil immer nur eine Funktion gleichzeitig läuft, braucht man keine Locks: Zwei Nachrichten können die Daten nie gleichzeitig verändern.

2.10 Module laden: require

`require("name")` lädt ein Modul, wie `import` in Python. `http` ist bei Node.js schon dabei, `ws` (WebSockets) kommt aus dem Paketverzeichnis npm. Die geschweiften Klammern sind wieder Destrukturierung: Aus dem Modul wird nur das Feld WebSocketServer genommen.

server.js, Zeilen 6–7

```
  6  const http = require("http");
  7  const { WebSocketServer } = require("ws");
```

3. Was ist ein WebSocket?

Normales HTTP ist wie ein Brief: Der Browser fragt, der Server antwortet, fertig. Für ein Spiel muss aber der Server dem Browser von sich aus sagen können: „Dein Gegner hat geschossen." Dafür gibt es **WebSockets** – eine dauerhaft offene Leitung, über die beide Seiten jederzeit Nachrichten schicken.

Ein WebSocket beginnt als normale HTTP-Anfrage mit dem Wunsch „Upgrade auf WebSocket". Nimmt der Server an, bleibt die Verbindung offen. Adressen beginnen mit `ws://` bzw. verschlüsselt `wss://`.

So sieht die Anfrage aus, mit der Bens Browser die Leitung öffnet

```
GET /spiele-ws?spiel=k7m2x9ab&rolle=gast&token=geheimben0000001 HTTP/1.1
Host: test.abc.de
Origin: https://test.abc.de
Upgrade: websocket
Connection: Upgrade
```

Im Browser ist das eine Zeile (`new WebSocket(adresse)`); in Node.js übernimmt das Paket `ws` die Server-Seite. Deshalb braucht nginx für diesen Pfad die besonderen Zeilen mit Upgrade und Connection.

4. server.js Schritt für Schritt

4.1 Einstellungen

server.js, Zeilen 9–17

```
  9  const PORT = Number(process.env.PORT) || 8080;
 10  const PFAD = process.env.WS_PFAD || "/spiele-ws";
 11  const MAX_SPIELE = Number(process.env.MAX_SPIELE) || 500;
 12  const MAX_NACHRICHT = 16 * 1024;              // Bytes je Nachricht
 13  const MAX_PRO_SEKUNDE = 30;                    // Nachrichten je Verbindung und Sekunde
 14  const SPIEL_ABLAUF = 24 * 60 * 60 * 1000;      // unbenutzte Spiele nach 24 Stunden löschen
 15
 16  const ROLLEN = ["gastgeber", "gast"];
 17  const anderer = rolle => (rolle === "gast" ? "gastgeber" : "gast");
```

`process.env` enthält die Umgebungsvariablen des Containers (im Dockerfile mit `ENV` gesetzt). `Number(...)` wandelt Text in eine Zahl um; ist die Variable nicht gesetzt, greift der Standardwert nach `||` (siehe 2.4). `24 * 60 * 60 * 1000` sind 24 Stunden in Millisekunden – Zeitangaben sind in JavaScript fast immer Millisekunden.

4.2 Die zentrale Datenstruktur

server.js, Zeilen 19–20

```
 19  // code -> { token: { gastgeber, gast }, socket: { gastgeber, gast }, zuletzt }
 20  const spiele = new Map();
```

Alle laufenden Spiele liegen in einer einzigen `Map` im Arbeitsspeicher. Schlüssel ist der Spielcode, Wert ein Objekt mit zwei Plätzen. So sieht ein Spiel aus, in dem Anna (Gastgeberin) und Ben (Gast) verbunden sind:

Inhalt der Map spiele (schematisch)

```
"k7m2x9ab" -> {
  token:   { gastgeber: "geheimanna000001", gast: "geheimben0000001" },
  socket:  { gastgeber: <Annas Verbindung>, gast: <Bens Verbindung> },
  zuletzt: 1761108318482          // Zeitpunkt der letzten Aktivität in ms
}
```

Das **token** ist ein zufälliger Geheimwert, den der Browser beim ersten Besuch erzeugt und speichert. Wer später mit demselben Code verbinden will, muss denselben Geheimwert vorzeigen – so kann niemand Fremdes einen Platz übernehmen. Das **socket** ist die offene Leitung, oder `null`, wenn das Gerät gerade nicht verbunden ist.

4.3 Hilfsfunktionen

server.js, Zeilen 22–36

```
 22  function log(...teile) {
 23    console.log(new Date().toISOString(), ...teile);
 24  }
 25
 26  // Eigene Nachrichten beginnen mit "!", damit ein Spieler sie nicht nachahmen kann:
 27  // von Spielern wird nur weitergereicht, was mit "{" beginnt.
 28  function sendeJson(ws, nachricht) {
 29    if (ws && ws.readyState === ws.OPEN) ws.send("!" + JSON.stringify(nachricht));
 30  }
 31
 32  function meldeStatus(spiel) {
 33    for (const rolle of ROLLEN) {
 34      sendeJson(spiel.socket[rolle], { typ: "_status", gegner:
    ↪ !!spiel.socket[anderer(rolle)] });
 35    }
 36  }
```

`...teile` sammelt alle übergebenen Argumente in einer Liste (Pythons `*args`), `...teile` beim Aufruf packt sie wieder aus. log schreibt so Zeitstempel plus beliebig viele Werte.

`sendeJson` schickt eigene Nachrichten des Servers. Das vorangestellte `!` ist ein Sicherheitsdetail: Der Server reicht von Spielern nur Text weiter, der mit { beginnt (Zeile 112). Eine Nachricht mit ! kann also nur vom Server selbst stammen – ein manipuliertes Spiel kann keine Server-Meldung wie „Spiel ist voll“ vortäuschen.

`meldeStatus` sagt beiden Spielern, ob ihr Gegner gerade verbunden ist. `!!spiel.socket[...]` macht aus „Verbindung oder null“ ein `true`/`false`.

4.4 Wer darf mitspielen? Die Funktion melde

server.js, Zeilen 38–53

```
 38  // Prüft die Anmeldung und liefert das Spiel oder einen Fehlergrund
 39  function melde(code, rolle, token) {
 40    let spiel = spiele.get(code);
 41    if (rolle === "gastgeber") {
 42      if (!spiel) {
 43        if (spiele.size >= MAX_SPIELE) return { fehler: "ausgelastet" };
 44        spiel = { token: { gastgeber: token, gast: null }, socket: { gastgeber: null, gast:
    ↪ null }, zuletzt: Date.now() };
 45        spiele.set(code, spiel);
 46        return { spiel };
 47      }
 48      return spiel.token.gastgeber === token ? { spiel } : { fehler: "belegt" };
 49    }
 50    if (!spiel) return { fehler: "unbekannt" };
 51    if (!spiel.token.gast) spiel.token.gast = token;
 52    return spiel.token.gast === token ? { spiel } : { fehler: "voll" };
 53  }
```

Diese Funktion ist das Herzstück der Zugangskontrolle. Sie liefert entweder das Spiel oder einen Fehlergrund. Statt sie nur zu beschreiben, hier ein Durchlauf aller Fälle – die Funktion ist unverändert aus server.js kopiert:

Beispiel: alle Fälle der Anmeldung

```
const MAX_SPIELE = 500;
const spiele = new Map();
function melde(code, rolle, token) {
  let spiel = spiele.get(code);
  if (rolle === "gastgeber") {
    if (!spiel) {
      if (spiele.size >= MAX_SPIELE) return { fehler: "ausgelastet" };
      spiel = { token: { gastgeber: token, gast: null }, socket: { gastgeber: null, gast: null
        ↪ }, zuletzt: Date.now() };
      spiele.set(code, spiel);
      return { spiel };
    }
    return spiel.token.gastgeber === token ? { spiel } : { fehler: "belegt" };
  }
  if (!spiel) return { fehler: "unbekannt" };
  if (!spiel.token.gast) spiel.token.gast = token;
  return spiel.token.gast === token ? { spiel } : { fehler: "voll" };
}

const zeige = (wer, ergebnis) => console.log(wer.padEnd(34), ergebnis.fehler || "ok");
zeige("Anna erstellt Spiel", melde("k7m2x9ab", "gastgeber", "anna-geheim"));
zeige("Ben kommt über den Link", melde("k7m2x9ab", "gast", "ben-geheim"));
zeige("Ben lädt die Seite neu", melde("k7m2x9ab", "gast", "ben-geheim"));
zeige("Cleo bekommt den Link auch", melde("k7m2x9ab", "gast", "cleo-geheim"));
zeige("Jemand gibt sich als Anna aus", melde("k7m2x9ab", "gastgeber", "falsch"));
zeige("Link mit falschem Code", melde("zzzzzzzz", "gast", "ben-geheim"));
```

Ausgabe

```
Anna erstellt Spiel                ok
Ben kommt über den Link            ok
Ben lädt die Seite neu             ok
Cleo bekommt den Link auch         voll
Jemand gibt sich als Anna aus      belegt
Link mit falschem Code             unbekannt
```

Fehler	Bedeutung	Was der Browser anzeigt
`belegt`	Spielcode gehört schon einem anderen Gastgeber	Neuer Code (nur bei frisch erstelltem Spiel)
`voll`	Gastplatz schon von einem anderen Gerät belegt	„Dieses Spiel hat schon zwei Spieler.“
`unbekannt`	Spielcode existiert (noch) nicht	„Spiel nicht gefunden …“ und neuer Versuch
`ausgelastet`	Schon 500 Spiele gleichzeitig	„Der Server ist ausgelastet …“

Warum der Gast nicht vorher eingeladen werden muss: Der erste Gast mit gültigem Code bekommt den freien Platz und „besitzt“ ihn ab dann über sein token. Deshalb gilt: Wer den Link zuerst öffnet, spielt mit.

4.5 Ein Mini-Webserver für den Gesundheitscheck

server.js, Zeilen 55–58

```
 55  const server = http.createServer((req, res) => {
 56    res.writeHead(req.url === "/gesund" ? 200 : 404, { "Content-Type": "text/plain;
   ↪ charset=utf-8" });
 57    res.end(req.url === "/gesund" ? "ok" : "nicht gefunden");
 58  });
```

`http.createServer` bekommt eine Funktion, die bei jeder normalen HTTP-Anfrage aufgerufen wird – mit der Anfrage (`req`) und der Antwort (`res`). Nur `/gesund` antwortet mit

„ok“. Das nutzt Docker: Die `HEALTHCHECK`-Zeile im Dockerfile ruft diese Adresse alle 30 Sekunden ab – daher das „(healthy)“ in docker compose ps.

server.js, Zeilen 60–60

```
60  const wss = new WebSocketServer({ noServer: true, maxPayload: MAX_NACHRICHT });
```

Der WebSocket-Server wird ohne eigenen Port angelegt (`noServer: true`), weil er sich die Verbindungen vom HTTP-Server übergeben lässt. `maxPayload` begrenzt jede Nachricht auf 16 KB; größere Nachrichten beenden die Verbindung sofort.

4.6 Der Türsteher: das upgrade-Ereignis

server.js, Zeilen 62–80

```
62  server.on("upgrade", (req, socket, kopf) => {
63    const url = new URL(req.url, "http://intern");
64    const code = url.searchParams.get("spiel") || "";
65    const rolle = url.searchParams.get("rolle") || "";
66    const token = url.searchParams.get("token") || "";
67
68    // Nur Seiten derselben Adresse dürfen sich verbinden, keine fremden Webseiten
69    const origin = req.headers.origin;
70    let originOk = false;
71    try { originOk = !!origin && new URL(origin).host === req.headers.host; } catch (e) {}
72
73    if (url.pathname !== PFAD || !originOk || !/^[a-z0-9]{8}$/.test(code) ||
74        !ROLLEN.includes(rolle) || !/^[a-z0-9]{16,64}$/.test(token)) {
75      socket.write("HTTP/1.1 400 Bad Request\r\nConnection: close\r\n\r\n");
76      socket.destroy();
77      return;
78    }
79    wss.handleUpgrade(req, socket, kopf, ws => verbunden(ws, code, rolle, token));
80  });
```

Bevor eine WebSocket-Leitung aufgemacht wird, prüft dieser Teil die Anfrage. Erst wenn alles stimmt, übergibt `handleUpgrade` die Verbindung an das ws-Paket, das dann `verbunden(...)` aufruft.

Prüfung	Warum
Pfad ist /spiele-ws	Nur die vorgesehene Adresse
Origin-Host = Host	Der Browser schickt im Origin-Kopf mit, von welcher Seite die Verbindung kommt. Nur die eigene Spiele-Seite darf verbinden – eine fremde Webseite kann den Dienst nicht benutzen.
Spielcode: 8 Zeichen a–z, 0–9	Kein Unsinn und keine überlangen Werte
Rolle: gastgeber oder gast	Nur die zwei Plätze
Token: 16–64 Zeichen a–z, 0–9	Gültiger Geheimwert

`new URL(req.url, "http://intern")` zerlegt die Adresse; der zweite Wert ist nur nötig, weil `req.url` keinen Rechnernamen enthält. So arbeitet das:

Beispiel

```
const url = new URL("/spiele-ws?spiel=k7m2x9ab&rolle=gast&token=geheimben0000001",
  ↪ "http://intern");
console.log(url.pathname);
console.log(url.searchParams.get("spiel"), url.searchParams.get("rolle"));
console.log(url.searchParams.get("fehlt") || "-");  // fehlt -> null -> "-"

const origin = "https://test.abc.de";
console.log(new URL(origin).host === "test.abc.de");
```

Ausgabe

```
/spiele-ws
k7m2x9ab gast
-
true
```

Das `try { ... } catch (e) {}` in Zeile 71 fängt den Fall ab, dass im Origin-Kopf Unsinn steht – `new URL` würde sonst einen Fehler werfen und den Dienst stören.

4.7 Eine Verbindung ist da: verbunden

server.js, Zeilen 82–96

```
 82  function verbunden(ws, code, rolle, token) {
 83    const { spiel, fehler } = melde(code, rolle, token);
 84    if (fehler) {
 85      sendeJson(ws, { typ: "_fehler", grund: fehler });
 86      ws.close(4000, fehler);
 87      return;
 88    }
 89
 90    // Dieselbe Person verbindet sich neu (z. B. nach Neuladen): alte Verbindung ersetzen
 91    const alt = spiel.socket[rolle];
 92    spiel.socket[rolle] = ws;
 93    if (alt) alt.close(4001, "ersetzt");
 94    spiel.zuletzt = Date.now();
 95    log("verbunden", code, rolle);
 96    meldeStatus(spiel);
```

Erst wird mit melde geprüft, ob das Gerät mitspielen darf. Wenn nicht, bekommt es den Fehlergrund und die Leitung wird geschlossen (4000 ist ein selbst gewählter Code; Codes ab 4000 sind für eigene Zwecke frei). Lädt jemand die Seite neu, kommt er mit einer neuen Verbindung zurück – die alte wird dann ersetzt und geschlossen.

server.js, Zeilen 98–116

```
 98    let fenster = Date.now();
 99    let anzahl = 0;
100    ws.lebt = true;
101    ws.on("pong", () => { ws.lebt = true; });
102
103    ws.on("message", (daten, binaer) => {
104      const jetzt = Date.now();
105      if (jetzt - fenster > 1000) { fenster = jetzt; anzahl = 0; }
106      if (++anzahl > MAX_PRO_SEKUNDE) {
107        ws.close(4002, "zu viele Nachrichten");
108        return;
109      }
110      if (binaer) return;
111      const text = daten.toString();
112      if (text[0] !== "{") return;
113      spiel.zuletzt = jetzt;
114      const ziel = spiel.socket[anderer(rolle)];
115      if (ziel && ziel.readyState === ziel.OPEN) ziel.send(text);
116    });
```

Für jede Verbindung werden ab Zeile 101 Ereignis-Funktionen angemeldet (siehe 2.9). Die Funktion für `message` läuft bei jeder eingehenden Nachricht und macht vier Dinge:

Zeilen	Was passiert
104–109	**Flutschutz:** zählt Nachrichten pro Sekunde; mehr als 30 → Verbindung trennen. Der Browser verbindet sich danach von selbst neu.
110–112	Nur Text, der mit { beginnt (also JSON-Objekte), wird weitergereicht.
113	Zeitpunkt merken – für das Aufräumen nach 24 Stunden.
114–115	**Weiterreichen:** an die Verbindung des anderen Spielers, falls der gerade verbunden ist. Ist er es nicht, geht die Nachricht verloren – das ist in Ordnung, weil die Spiele nach dem Wiederverbinden ihren Stand abgleichen.

`++anzahl` erhöht zuerst und vergleicht dann (wie in C oder Java). So funktioniert der Flutschutz im Kleinen:

Beispiel

```
const MAX_PRO_SEKUNDE = 30;
let fenster = 0, anzahl = 0;

function nachrichtKommt(jetzt) {           // jetzt = Zeit in ms
  if (jetzt - fenster > 1000) { fenster = jetzt; anzahl = 0; }
  return ++anzahl > MAX_PRO_SEKUNDE ? "trennen" : "ok";
}

const ergebnisse = [];
// 31 Nachrichten in derselben Sekunde:
for (let i = 0; i < 31; i++) ergebnisse.push(nachrichtKommt(5000));
console.log("Nachricht 30:", ergebnisse[29], "| Nachricht 31:", ergebnisse[30]);
console.log("Eine Sekunde später:", nachrichtKommt(6001));
```

Ausgabe

```
Nachricht 30: ok | Nachricht 31: trennen
Eine Sekunde später: ok
```

server.js, Zeilen 118–127

```
118    ws.on("close", () => {
119      if (spiel.socket[rolle] !== ws) return;
120      spiel.socket[rolle] = null;
121      spiel.zuletzt = Date.now();
122      log("getrennt", code, rolle);
123      meldeStatus(spiel);
124    });
125
126    ws.on("error", () => {});
127  }
```

Schließt sich eine Leitung, wird der Platz frei (`null`) und der Gegner informiert. Die Prüfung in Zeile 119 ist wichtig: Wurde die Verbindung wegen Neuladens schon ersetzt (Zeile 93), darf ihr `close`-Ereignis die neue Verbindung nicht austragen. Das leere `error`-Ereignis verhindert, dass ein Netzwerkfehler den ganzen Dienst beendet – unbehandelte Fehler-Ereignisse würden Node.js abstürzen lassen.

4.8 Aufräumen alle 30 Sekunden

server.js, Zeilen 129–140

```
129  // Tote Verbindungen erkennen und alte Spiele aufräumen
130  setInterval(() => {
131    for (const ws of wss.clients) {
132      if (!ws.lebt) { ws.terminate(); continue; }
133      ws.lebt = false;
134      ws.ping();
135    }
136    const grenze = Date.now() - SPIEL_ABLAUF;
137    for (const [code, spiel] of spiele) {
138      if (!spiel.socket.gastgeber && !spiel.socket.gast && spiel.zuletzt < grenze)
  ↪ spiele.delete(code);
139    }
140  }, 30000);
```

`setInterval` ruft die Funktion alle 30 000 ms auf. Sie erledigt zwei Dinge:

Tote Verbindungen erkennen: Fällt bei einem Handy das WLAN weg, merkt der Server das nicht sofort. Deshalb schickt er jedem ein `ping`; ein lebendiges Gerät antwortet automatisch mit `pong` (Zeile 101 setzt dann `lebt = true`). Wer bis zum nächsten Durchlauf nicht geantwortet hat, wird mit `terminate` hart getrennt.

Alte Spiele vergessen: Spiele ohne verbundene Spieler, die seit 24 Stunden unbenutzt sind, werden aus der Map gelöscht.

4.9 Start und sauberes Beenden

server.js, Zeilen 142–150

```
142  server.listen(PORT, () => log(`Weiterleitungsdienst läuft auf Port ${PORT}, Pfad
  ↪ ${PFAD}`));
143
144  for (const signal of ["SIGTERM", "SIGINT"]) {
145    process.on(signal, () => {
146      for (const ws of wss.clients) ws.close(1001, "Neustart");
147      server.close(() => process.exit(0));
148      setTimeout(() => process.exit(0), 2000).unref();
149    });
150  }
```

`server.listen` öffnet den Port; die Funktion wird aufgerufen, sobald er bereit ist. Ab hier läuft das Programm „ewig“ weiter, weil es auf Ereignisse wartet.

Wenn Docker den Container stoppt (z. B. bei `docker compose restart spiele-relay`), schickt es das Signal `SIGTERM`. Der Dienst schließt dann alle Leitungen höflich mit Code 1001 („Server geht weg“), damit die Browser sofort neu verbinden. `.unref()` bedeutet: Dieser Notfall-Zeitgeber allein soll das Programm nicht am Leben halten.

5. Ein Spiel aus Sicht des Dienstes

Zum Ausprobieren gibt es ein kleines Testprogramm, das drei Spieler simuliert: Anna erstellt ein Spiel, Ben tritt bei und schießt auf Feld D5, Cleo versucht danach ebenfalls beizutreten. Es benutzt das Paket ws als Client und läuft direkt im Container, sodass auf dem Pi nichts installiert werden muss.

Datei probe.js

```
const WebSocket = require("ws");

const basis = "ws://127.0.0.1:8080/spiele-ws?spiel=k7m2x9ab";
const optionen = { origin: "http://127.0.0.1:8080" };

function spieler(name, rolle, token) {
  const ws = new WebSocket(`${basis}&rolle=${rolle}&token=${token}`, optionen);
  ws.on("message", daten => console.log(`${name} empfängt: ${daten}`));
  return ws;
}

const anna = spieler("Anna", "gastgeber", "geheimanna000001");

setTimeout(() => {
  const ben = spieler("Ben ", "gast", "geheimben0000001");
  ben.on("open", () => {
    setTimeout(() => ben.send(JSON.stringify({ typ: "schuss", runde: 1, x: 3, y: 4 })), 200);
  });
}, 300);

setTimeout(() => spieler("Cleo", "gast", "geheimcleo000001"), 900);
setTimeout(() => process.exit(0), 1500);
```

Auf dem Pi speicherst du das als `probe.js` im Ordner deiner docker-compose.yml und führst aus:

Befehl auf dem Pi

```
docker compose exec -T spiele-relay node - < probe.js
```

Ausgabe (echter Lauf)

```
Anna empfängt: !{"typ":"_status","gegner":false}
Anna empfängt: !{"typ":"_status","gegner":true}
Ben  empfängt: !{"typ":"_status","gegner":true}
Anna empfängt: {"typ":"schuss","runde":1,"x":3,"y":4}
Cleo empfängt: !{"typ":"_fehler","grund":"voll"}
```

Gleichzeitig im Protokoll: docker compose logs spiele-relay (ohne Zeitstempel)

```
verbunden k7m2x9ab gastgeber
verbunden k7m2x9ab gast
getrennt k7m2x9ab gastgeber
getrennt k7m2x9ab gast
```

Schön zu sehen: Server-Nachrichten beginnen mit !, Bens Schuss kommt bei Anna genau so an, wie Ben ihn geschickt hat, und Cleo wird abgewiesen. Im Protokoll stehen nur Spielcode und Rolle, keine Spielzüge.

Zum Experimentieren: Ersetze in `probe.js` bei Ben den Geheimwert durch einen anderen und starte erneut – dann bekommt Ben `voll`. Oder entferne `optionen` beim Verbindungsaufbau: Dann fehlt der Origin-Kopf, der Türsteher (4.6) lehnt ab, und das Testprogramm bricht mit `Error: Unexpected server response: 400` ab. Das Testprogramm benutzt den Spielcode k7m2x9ab; laufende echte Spiele stört es nicht.

6. Die anderen Dateien

6.1 package.json – die Projektbeschreibung

```
{
  "name": "spiele-relay",
  "version": "1.0.0",
  "private": true,
  "description": "Weiterleitungsdienst für Schiffe versenken im Heimnetz",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "ws": "8.21.3"
  }
}
```

Entspricht ungefähr Pythons `requirements.txt` plus Metadaten. Wichtig ist nur `dependencies`: Das Projekt braucht das Paket `ws` in genau Version 8.21.3. `npm` ist der Paketmanager von Node.js (wie `pip`). Ohne `^` oder `~` vor der Versionsnummer wird keine neuere Version automatisch genommen.

6.2 package-lock.json – der Fingerabdruck

Diese Datei erzeugt npm automatisch. Sie hält fest, welche Version genau installiert wird und mit welcher Prüfsumme (`integrity`). Weicht der Download auch nur um ein Byte ab, bricht die Installation ab. So bekommt der Pi garantiert denselben Code, der hier getestet wurde.

6.3 Dockerfile – die Bauanleitung für den Container

```
FROM node:22-alpine

WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY server.js ./

USER node
ENV PORT=8080 WS_PFAD=/spiele-ws
EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=3s CMD wget -qO- http://127.0.0.1:8080/gesund || exit 1
CMD ["node", "server.js"]
```

Zeile	Bedeutung
`FROM node:22-alpine`	Startet mit einem fertigen, kleinen Linux (Alpine) mit Node.js 22 – passend für den Pi (ARM 64-Bit).
`WORKDIR /app`	Arbeitsordner im Container.
`COPY package*.json …`	Erst nur die Paketliste kopieren …
`RUN npm ci --omit=dev`	… und daraus genau die Pakete aus package-lock.json installieren. ci = „clean install“, streng nach Lockfile.
`COPY server.js ./`	Dann das Programm. Die Reihenfolge spart Zeit: Ändert sich nur server.js, nimmt Docker die Pakete aus dem Zwischenspeicher.
`USER node`	Das Programm läuft nicht als Administrator, sondern als eingeschränkter Benutzer.
`ENV …`	Setzt die Umgebungsvariablen, die server.js über process.env liest (4.1).

Zeile	Bedeutung
`HEALTHCHECK …`	Docker fragt alle 30 Sekunden /gesund ab (4.5).
`CMD ["node", "server.js"]`	Der Befehl beim Start des Containers.

6.4 Die Gegenseite im Browser

Zum Vergleich der wichtigste Teil aus schiffe-versenken.html – der Browser hat WebSockets eingebaut, es braucht kein Paket:

Vereinfacht aus schiffe-versenken.html

```
const socket = new WebSocket("wss://test.abc.de/spiele-ws?spiel=...&rolle=...&token=...");

socket.onmessage = ereignis => {
  if (ereignis.data[0] === "!") {
    // Nachricht vom Server: _status (Gegner da?) oder _fehler (voll, unbekannt, …)
  } else {
    // Nachricht vom Gegner: hallo, bereit, schuss, ergebnis, aufdeckung, revanche
  }
};

socket.onclose = () => {
  // nach 1, 2, 4, 8, dann alle 15 Sekunden neu verbinden
};

socket.send(JSON.stringify({ typ: "schuss", runde: 1, x: 3, y: 4 }));
```

Statt `.on("message", …)` wie in Node.js weist man im Browser eine Funktion an `onmessage` zu – dasselbe Prinzip der Ereignisse.

7. Kleines Wörterbuch

Begriff	Erklärung
Node.js	Programm, das JavaScript außerhalb des Browsers ausführt
npm	Paketmanager von Node.js, wie pip bei Python
WebSocket	Dauerhaft offene Leitung zwischen Browser und Server; beide Seiten können jederzeit senden
Ereignis / Callback	Funktion, die Node.js später aufruft, wenn etwas passiert (.on(…), setTimeout)
Pfeilfunktion	Kurzschreibweise x => ausdruck für eine Funktion
Map	Wörterbuch mit get, set, delete, size
Origin	Kopfzeile, mit der der Browser verrät, von welcher Webseite eine Verbindung kommt
Token	Zufälliger Geheimwert, mit dem ein Gerät seinen Platz im Spiel belegt
Ping / Pong	Lebenszeichen-Abfrage bei WebSockets
Umgebungsvariable	Einstellung, die von außen (Docker) an das Programm übergeben wird: process.env